

MQTT

Miguel Leitão, 2025

What does “MQTT” stand for?

Formerly: **MQ Telemetry Transport**,.

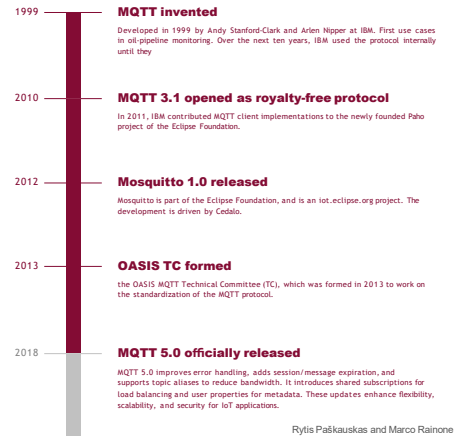
Also known as: **M**essage **Q**ueue **T**elemetry **T**ransport.

Now: **MQTT**

- no longer an acronym.
- simply the name of the protocol.

History and Evolution

MQTT is a lightweight, publish/subscribe messaging protocol designed for constrained devices and unreliable networks.



3

Why Use MQTT?

MQTT is used because it's simple, efficient, and works well even when the Internet connection is slow or unstable.

Asynchronous communication: Clients can publish data without worrying about whether subscribers are available, as the broker handles message routing. It allows devices to send and receive messages without waiting for an immediate response.

Low overhead and reduced power consumption: With a small packet size and a lightweight control mechanism, MQTT is efficient for transmitting data.

Support for Intermittent Connections: Devices in constrained environments often lose network connectivity. MQTT supports persistent sessions to handle this gracefully.

4

MQTT Ecosystem

Use cases: Home automation, industrial IoT, autonomous vehicles, smart cities.

Commercial brokers: HiveMQ, EMQX

Supported by cloud services: AWS IoT, Azure IoT Hub, Google Cloud IoT Core, IBM Watson, Oracle IoT

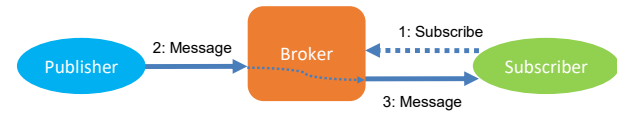
MQTT-like protocols: Apache Kafka (LinkedIn), Pulsar, ActiveMQ, ...

Client APIs: Mosquitto, Paho

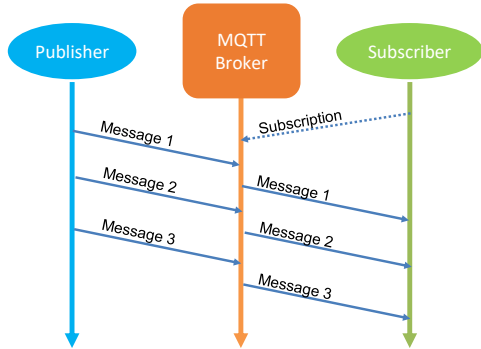
Languages: C, Python, Javascript, ...

Broker Software: Mosquitto

Publish / Subscribe model

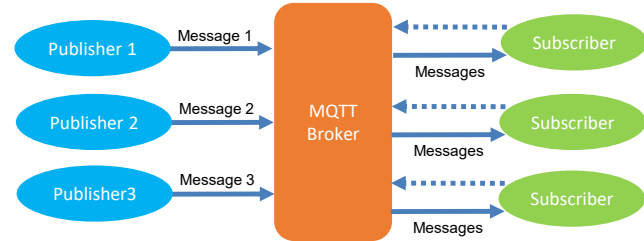


Publish / Subscribe



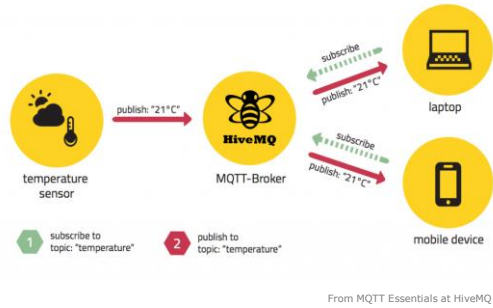
7

Multiple Publishers / Multiple Subscribers

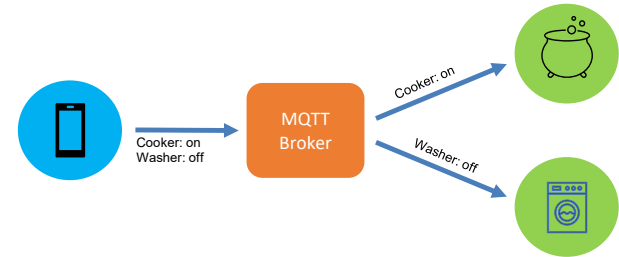


8

MQTT Example: Sensor



MQTT Example: Control



Clients

Publishers are **Senders**.

Subscribers are **Receivers**.

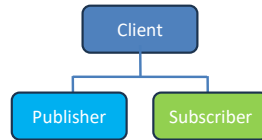
Publishers + Subscribers = Clients

A single device can act as a **Publisher** and a **Subscriber**.

The **broker**'s IP and port must be known by **Clients**.

The **Clients** do not know each other.

The **Clients** do not need to be running at the same time.



MQTT Broker

The **broker** is primarily responsible for **receiving** all messages, filtering them, decide who is interested in it and then **sending** the message to all subscribed clients. (From HiveMQ MQTT Essentials)

May authenticate and authorize clients.

Maintains sessions and missed messages.

Maintains active messages in a hierarchical structure of topics.

Message Transport

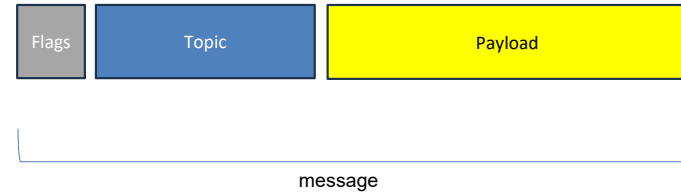
Messages between **Clients** are transported in two steps:

1. **Publication:** From a **Publisher** to a **Broker**.
2. **Consuming:** From a **Broker** to a **Subscriber**.

Each transport step is carried by within a **TCP** or **TLS** session.

A single **TCP** connection can transport multiple **MQTT** messages.

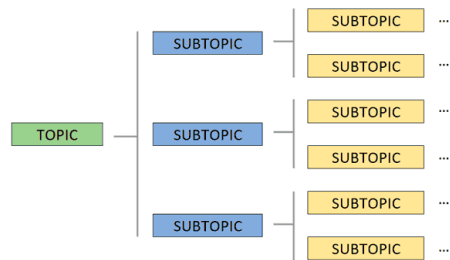
MQTT Message



MQTT Topic

Identifies the Message

Topics are hierarchical organized: **/MainTopic/SubTopic/.../SubSubTopic/Item**



15

MQTT Topic

Topics are vectors of ASCII chars

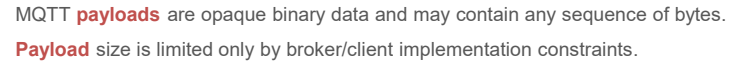
Topics are written as sequences of **/subtopic** levels.

Examples:

- /MyHouse/livingRoom/sensor/1/temperature
- /MyHouse/livingRoom/sensor/1/pressure
- /MyHouse/bedRoom/sensor/3/temperature
- /MyHouse/bedRoom/sensor/4/humidity

16

Message Payload



Prefer human-readable encodings (ASCII/UTF-8).

Prefer structured, machine-parseable formats.

Keep payloads atomic when possible (single logical data item per message).

Use well-established serialization formats such as JSON, CSV, ...

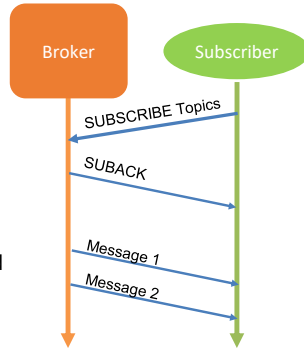
MQTT Subscription

A client can **subscribe** a set of **Topics** by sending a SUBSCRIBE message.

Topics can be:

- A **single** Topic
- A **list** of Topics
- A **wildcard set** of Topics

Each subscribe message is acknowledged by a SUBACK confirmation.



MQTT Topic wildcard

A subscriber can use **wildcards** to subscribe a set of topics.

- /MyHouse/garage/sensor/+/temperature
- /MyHouse/livingroom/#
- /MyHouse/bedroom/+3/temperature
- /MyHouse+/sensor/4/humidity
- /MyHouse/#
- /#

- + : Single level wildcard.
can appear anywhere in the topic string.
- # : Multilevel wildcard.
must appear at the end of the topic.

MQTT Publish/Subscribe Paradigm vs HTTP Request/Response

Feature	MQTT (Publish/Subscribe)	HTTP (Request/Response)
Communication Model	Decoupled: many-to-many communication with a broker	Direct: one-to-one communication
Message Flow	messages sent to topics, received by subscribers via broker	client requests data from a server, receives a response
Statefulness	Long-lived connections, maintains state	Stateless, each request is independent

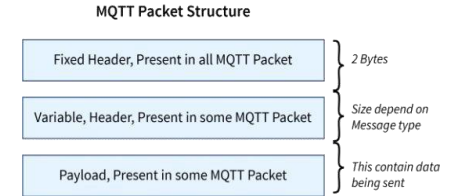
MQTT Protocol

MQTT Client and Broker “talking”



MQTT Packet Structure

- **Key components:** Fixed header, variable header, and payload.
- **Common packet types:** CONNECT, PUBLISH, SUBSCRIBE, PINGREQ, DISCONNECT.



SUBSCRIBE	subscribe (client)	
PUBLISH	publish (client and broker)	
DISCONNECT	disconnect	
PINGREQ	ping request	
PINGRESP	ping response	
CONNACK	connection acknowledged	
SUBACK	subscribe acknowledged	
PUBACK	publish acknowledged	
PUBREC	publish request	
PUBREL	publish release	
PUBCOMP	publish completed	

25

Core MQTT Concepts

Publish: The action of sending a message to a specific topic on the broker.

Subscribe: The action of a client registering to receive messages from a specific topic.

Topic: A string used to organize and categorize messages, acting as a routing key for message distribution.

Broker: Central server that manages communication between clients by routing messages from publishers to subscribers.

Client: Any device or application that connects to the broker to publish or subscribe to topics.

QoS (Quality of Service): Defines the level of guarantee for message delivery (0, 1, or 2).

Retain Flag: Option to store the last message sent to a topic for future subscribers.

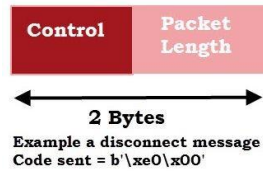
Will Message: A message defined by a client that the broker will send if the client unexpectedly disconnects.

Clean Session: An option to specify whether a client's subscription and message queue should be cleared when it disconnects.

Session Persistence: The broker can retain client subscriptions and undelivered messages between disconnections.

26

MQTT minimum packet



mosquitto_sub example

```
> mosquitto_sub -h localhost -p 1883 -t '#'
```

```
1729511061: New connection from 127.0.0.1:44770 on port 1883.
1729511061: New client connected from 127.0.0.1:44770 as
auto-75A73584-46A3-85F6-BB86-F63D49823F2B (p2, c1, k60).
1729511061: No will message specified.
1729511061: Sending CONNACK to auto-75A73584-46A3-85F6-BB86-F63D49823F2B (0, 0)
1729511061: Received SUBSCRIBE from auto-75A73584-46A3-85F6-BB86-F63D49823F2B
1729511061: # (QoS 0)
1729511061: auto-75A73584-46A3-85F6-BB86-F63D49823F2B 0 #
1729511061: Sending SUBACK to auto-75A73584-46A3-85F6-BB86-F63D49823F2B
1729511121: Received PINGREQ from auto-75A73584-46A3-85F6-BB86-F63D49823F2B
1729511121: Sending PINGRESP to auto-75A73584-46A3-85F6-BB86-F63D49823F2B
...
```

mosquitto_pub example

```
> mosquitto_pub -h localhost -p 1883 -t 'hello' -m 'world'
```

```
1729511519: New client connected from 127.0.0.1:60848 as
auto-791F0260-5704-A25C-7820-3CAD54656FEB (p2, c1, k60).
1729511519: No will message specified.
1729511519: Sending CONNACK to auto-791F0260-5704-A25C-7820-3CAD54656FEB (0, 0)
1729511519: Received PUBLISH from auto-791F0260-5704-A25C-7820-3CAD54656FEB (d0, q0, r0,
m0, 'hello', ... (5 bytes))
1729511519: Sending PUBLISH to auto-75A73584-46A3-85F6-BB86-F63D49823F2B (d0, q0, r0,
m0, 'hello', ... (5 bytes))
1729511519: Received DISCONNECT from auto-791F0260-5704-A25C-7820-3CAD54656FEB
1729511519: Client auto-791F0260-5704-A25C-7820-3CAD54656FEB disconnected.
```

29

Quality of Service (QoS) Levels

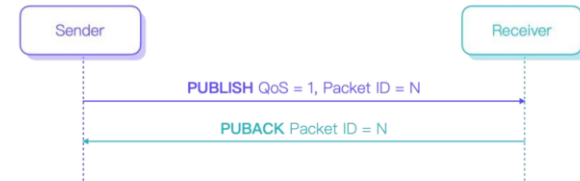
QoS Level	It's like...	When to Use
QoS 0 At most once	Like sending a postcard: no guarantee it will arrive, and no follow-up.	For non-critical, idempotent and low latency data, such interactive commands and periodic sensor updates.
QoS 1 At least once	Guarantees message delivery but may result in duplicates	Idempotent commands and sensor states.
QoS 2 Exactly once	Sending a package with a signature: delivered once, and only once, no duplicates.	For critical and non-idempotent messages

30

QoS 0



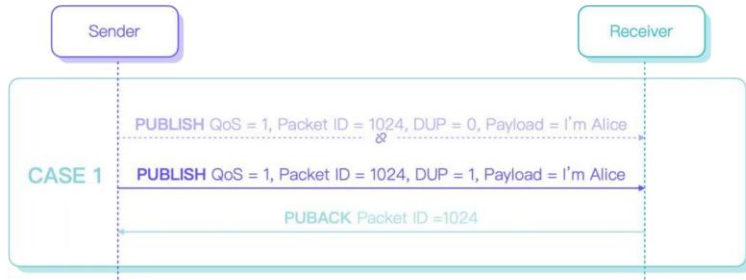
QoS 1 – Normal operation



31

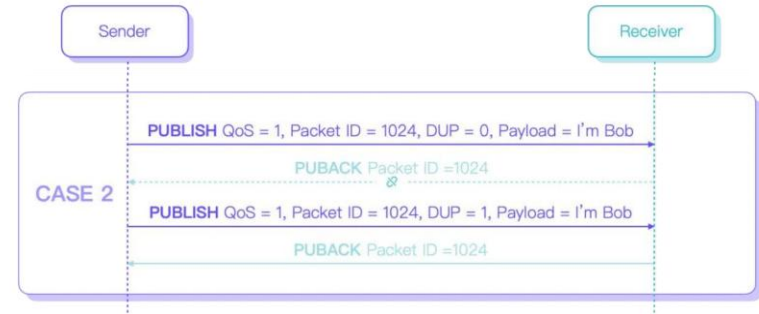
32

QoS 1 – DataPacket Lost



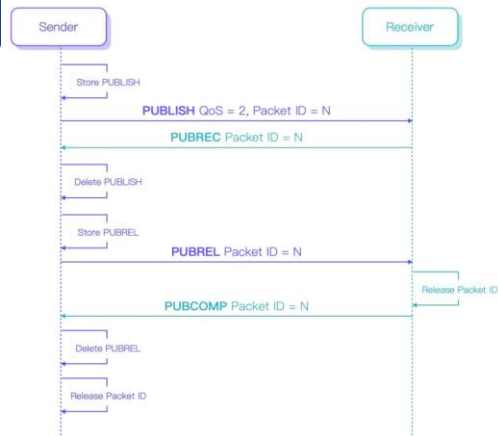
33

QoS 1 – Acknowledge Lost



34

QoS 2



35

Mosquitto_sub with QoS

> **mosquitto_sub -q 1 -h localhost -p 1883 -t '#'**

```

1729512609: New connection from 127.0.0.1:36906 on port 1883.
1729512609: New client connected from 127.0.0.1:36906 as
auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4 (p2, c1, k60).
1729512609: No will message specified.
1729512609: Sending CONNACK to auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4 (0, 0)
1729512609: Received SUBSCRIBE from auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4
1729512609: # (QoS 1)
1729512609: auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4 1 #
1729512609: Sending SUBACK to auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4
...
  
```

36

mosquitto_pub with QoS

```
> mosquitto_pub -q 1 -h localhost -p 1883 -t 'hello' -m 'world'
```

```
1729512717: New connection from 127.0.0.1:46972 on port 1883.
1729512717: New client connected from 127.0.0.1:46972 as auto-
35FE21EB-AE06-DF79-5138-A11159EAEA94 (p2, c1, k60).
1729512717: No will message specified.
1729512717: Sending CONNACK to auto-35FE21EB-AE06-DF79-5138-A11159EAEA94 (0, 0)
1729512717: Received PUBLISH from auto-35FE21EB-AE06-DF79-5138-A11159EAEA94 (d0, q1, r0,
m1, 'hello', ... (5 bytes))
1729512717: Sending PUBLISH to auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4 (d0, q1, r0,
m1, 'hello', ... (5 bytes))
1729512717: Sending PUBACK to auto-35FE21EB-AE06-DF79-5138-A11159EAEA94 (m1, rc0)
1729512717: Received PUBACK from auto-ED3A5FED-675B-366D-EC69-0B6BD83796F4 (Mid: 1, from auto-
ED3A5FED-675B-366D-EC69-0B6BD83796F4 (Mid: 1, RC:0))
1729512717: Received DISCONNECT from auto-35FE21EB-AE06-DF79-5138-A11159EAEA94
1729512717: Client auto-35FE21EB-AE06-DF79-5138-A11159EAEA94 disconnected.
```

37

Mosquitto_sub with QoS 2

```
> mosquitto_sub -q 2 -h localhost -p 1883 -t '#'
```

```
1729513207: New connection from 127.0.0.1:49236 on port 1883.
1729513207: New client connected from 127.0.0.1:49236 as
auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 (p2, c1, k60).
1729513207: No will message specified.
1729513207: Sending CONNACK to auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 (0, 0)
1729513207: Received SUBSCRIBE from auto-E41AFA10-1E5D-E733-F44E-5750B87AF619
1729513207: # (QoS 2)
1729513207: auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 2 #
1729513207: Sending SUBACK to auto-E41AFA10-1E5D-E733-F44E-5750B87AF619
...
```

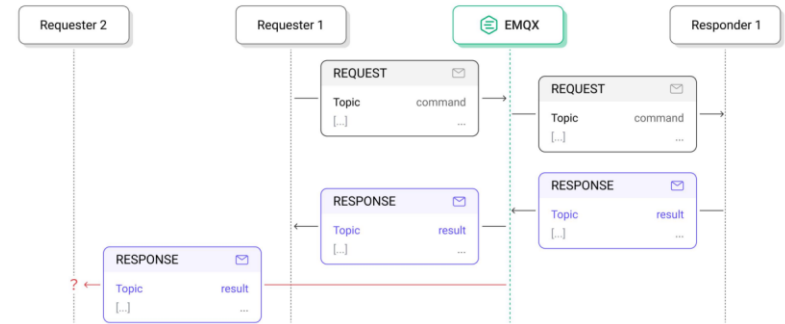
38

QoS 2: Publish

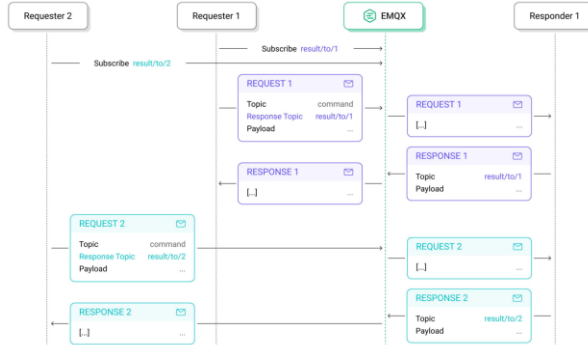
```
> mosquitto_pub -q 2 -h localhost -p 1883 -t 'hello' -m 'world'
```

```
1729513374: New connection from 127.0.0.1:50520 on port 1883.
1729513374: New client connected from 127.0.0.1:50520 as
auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 (p2, c1, k60).
1729513374: No will message specified.
1729513374: Sending CONNACK to auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 (0, 0)
1729513374: Received PUBLISH from auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 (d0, q2, r0,
m1, 'hello', ... (5 bytes))
1729513374: Sending PUBREC to auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 (m1, rc0)
1729513374: Received PUBREL from auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 (Mid: 1)
1729513374: Sending PUBLISH to auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 (d0, q2, r0,
m1, 'hello', ... (5 bytes))
1729513374: Sending PUBCOMP to auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 (m1)
1729513374: Received DISCONNECT from auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20
1729513374: Client auto-125DDDBC-DB9D-905F-78EF-87F602FFEE20 disconnected.
1729513374: Received PUBREC from auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 (Mid: 1)
1729513374: Sending PUBREL to auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 (m1)
1729513374: Received PUBCOMP from auto-E41AFA10-1E5D-E733-F44E-5750B87AF619 (Mid: 1, RC:0)
```

Request / Response

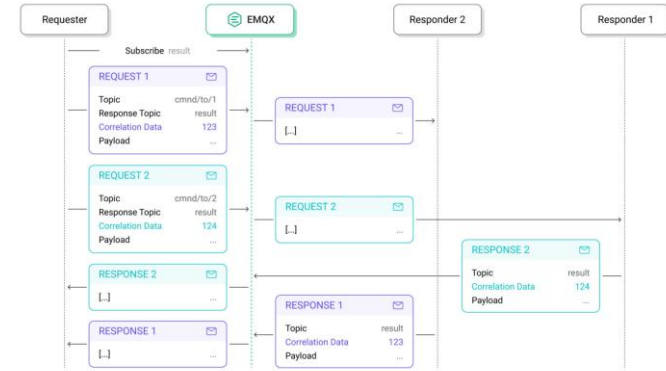


Request / Response



41

MQTT 5.0 - Request / Response



42

Retained Messages

Retained Messages ensure that the latest message is always available to new subscribers, even if they weren't connected when it was first sent.

It's like leaving a sticky note with the most recent information for anyone who checks later.

Example:

A weather station that publishes temperature data.

If a new device (like a phone or dashboard) subscribes to the weather topic, without retained messages, it would only get new updates and miss the latest temperature.

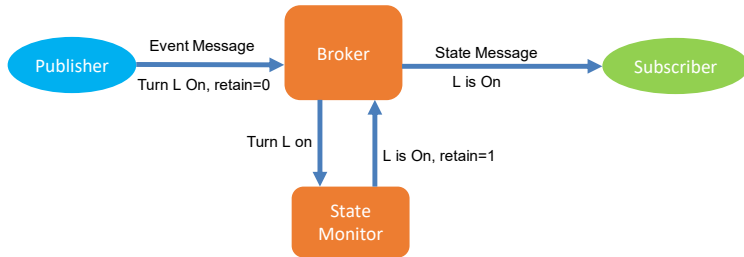
With a retained message, the MQTT broker keeps the most recent temperature data and immediately sends it to any new subscriber.

Retained Logs / Database

/House/MainDoor/Event/Log/2025/07/21/07/31	Open
/House/MainDoor/Event/Log/2025/07/21/07/32	Close
/House/MainDoor/Event/Log/2025/07/21/16/53	Open
/House/MainDoor/Event/Log/2025/07/21/16/34	Close

/ISEP/Student/19712345	{name="xxx", birth="ddd", ... }
/ISEP/Student/19712346	{name="xxx", birth="ddd", ... }
/ISEP/Student/19712365	{name="xxx", birth="ddd", ... }
/ISEP/Student/19712367	{name="xxx", birth="ddd", ... }
/ISEP/Student/19712381	{name="xxx", birth="ddd", ... }

State Cache



45

Last Will and Testament (LWT)

The Last Will and Testament (LWT) in MQTT is like a backup plan for when a device unexpectedly goes offline.

- The LWT is a special message the device prepares in advance (in-live).
- LWT message is send to the Broker and stored there permanently (life-long).
- If the device suddenly loses its connection, the MQTT broker automatically sends LWT message to all subscribed devices.
- For example, in a smart home, if a sensor stops working, the LWT could notify the system to alert the user or switch to a backup sensor. It helps keep everything running smoothly even when a device fails

46

MQTT Security

1. **Username and Password:** Devices can use a username and password to connect, like logging into an account. This helps ensure that only authorized devices can communicate.
2. **Encryption (TLS/SSL):** MQTT can use encryption (like HTTPS on websites) to keep data safe while it's being sent, so no one can read or change the messages during transmission.
3. **Access Control:** The broker can control who can publish or subscribe to topics, making sure only the right devices can send or receive specific messages.

47

Feature	MQTT	HTTP	CoAP	AMQP
Communication Model	Publish/Subscribe (asynchronous)	Request/Response (synchronous)	Request/Response (synchronous), Observe	Publish/Subscribe Message Queue (asynchronous)
Transport Protocol	TCP	TCP (HTTP/3 uses QUIC)	UDP (TCP is possible)	TCP
Overhead	Low	High	Very Low	Moderate to High
Message Delivery	QoS 0, 1, 2 (reliable with options)	TCP-level reliability, no QoS	Basic reliability with UDP	Highly reliable (transactional messaging)
Security	TLS	HTTPS (TLS)	DTLS	TLS, SASL

48

Feature/ Protocol	MQTT	HTTP	CoAP	AMQP
Statefulness	Session persistence for reconnecting clients	Stateless by design	Stateless, with optional Observe feature	Stateful
Scalability	High, designed for IoT environments	High, but overhead increases	Very High, designed for constrained devices	High, for large enterprise applications
Use Cases	IoT, remote sensors, constrained devices	Web applications, REST APIs, document exchange	Low-power IoT (smart grids, home automation)	Enterprise message brokering, financial systems

49

Feature / Protocol	MQTT	HTTP	CoAP	AMQP
QoS/ Delivery Guarantees	QoS levels ensure message delivery guarantees	No built-in QoS beyond TCP reliability	Basic retransmissions (confirmable messages)	Guarantees through transactional messaging
Message Size	Small	Large	Small	Large
Latency	Low (ideal for low-latency environments)	High for IoT (due to overhead)	Low	Moderate

50

When to Choose Each Protocol

MQTT: Ideal for low-bandwidth, high-latency environments where devices need to conserve power, such as IoT devices (sensors, actuators, smart home systems). Suitable when asynchronous, decoupled communication is needed, such as in telemetry or distributed control systems.

HTTP: Best for web applications and REST-based APIs where resource exchange and direct request-response interactions are required (e.g., file sharing, data retrieval from a server). Inefficient for IoT scenarios due to its high overhead and statelessness.

CoAP: Designed for constrained IoT devices that need efficient, low-overhead communication. Ideal for sensor networks, smart energy management, and low-power wireless communication.

AMQP: Preferred in enterprise-level applications where reliable, secure, and transactional messaging is required (e.g., financial institutions, message queuing systems, distributed systems). Suitable for complex routing and store-and-forward messaging scenarios where full delivery guarantees and reliability are crucial.